

Tracker — Platform Documentation

Based on source code review of `trackerApi` (Express + GraphQL + Prisma/SQLite) and `trackerProject` (Vite + React Router) as of current master branches.

Architecture Overview

The platform is split into two repos:

- trackerApi** — Node/TypeScript backend. Express server with Apollo GraphQL, Prisma ORM over SQLite. Handles all data persistence and business logic.
- trackerProject** — Vite + React Router frontend (PWA-ready, with manifest and service worker in dist). Communicates with the API exclusively via GraphQL mutations/queries.

Auth is JWT-based (register/login mutations return a token, stored client-side and sent as a Bearer header).

Core Concepts

Priority (P / S / O / B)

A four-quadrant urgency/importance matrix applied to Actions and Projects:

Code	Name	Meaning
P	Primary	High urgency + high importance
S	Secondary	Low urgency + high importance
O	Outsource	High urgency + low importance
B	Bucket List	Low urgency + low importance

The Hierarchy

The platform models goals as a tree of intent, with execution tracked at the Action level:

Goal (or Goal Group)

- └─ Milestone (ordered steps toward a goal)
 - └─ Project (a defined body of work)
 - └─ Action (a schedulable unit of work)

Goal Group

- └─ Milestone
 - └─ Goal (child goal – recursive nesting)

Goal Groups are a special goal variant where children are other Goals rather than Projects. They're for organizing clusters of related goals under a common theme.

Milestones represent ordered progress checkpoints within a goal. One milestone can be marked as the "last" (completion) milestone. They have a `doa` (Definition of Achieved) field and an optional `predictionDate`. Milestones are drag-reorderable.

Intervals and Routines (recurring activities) can be linked to a goal, a milestone, or a project — or left standalone.

Modules

1. Goals

What it does: Create and manage goals, optionally organized into a hierarchy (goal groups with child goals). Goals have a Definition of Done (`dod`) field, start/end dates, and a computed progress status based on their projects' completion state.

Key features:

- Create goals as regular goals or goal groups
- Nest goals inside goal groups (directly or under milestones)
- Add milestones to a goal and reorder them via drag-and-drop
- Link an existing goal as a child of another goal or milestone
- Mark a milestone as "last" (completion indicator)
- Inline-edit the goal title and DoD without navigating away
- Progress bar based on project completion across all milestones
- Goal status computed from project states (in-progress, done, not-started)

Data model fields:

- `title`, `dod` (Definition of Done)

- `isGoalGroup` (bool)
- `parentGoalId` / `parentMilestoneId` (optional hierarchy links)
- `startDate`, `endDate`

Status: Functional. The `dod` field exists in the data model and UI but the *clarity check* feature (an AI-assisted or structured check of whether the DoD is well-defined) is not yet implemented — this is noted as a known missing feature.

2. Milestones

What it does: Milestones are progress checkpoints belonging to a goal. They carry a `doa` (Definition of Achieved) field, an optional `predictionDate`, and an ordering index.

Key features:

- Create milestones under a goal
 - Optional prediction date (informational only, no effect on logic)
 - Drag-reorder milestones within a goal
 - Mark one milestone as "last"
 - Within a Goal Group: milestones hold child goals instead of projects
 - When deleting a milestone, user chooses to: delete its projects, move them to the goal, or move them to another milestone
-

3. Projects

What it does: Projects represent a body of work sitting under a goal or milestone. They hold Actions and can be linked to Intervals.

Key features:

- Create projects linked to a goal or milestone (one at a time, enforced)
- `dod` (Definition of Done), `type` (individual / team), `priority`
- Start and end dates
- Add actions directly to a project
- View project completion status based on action completion
- Projects are sorted chronologically within the goal's "story" view

Data model fields:

- `title`, `dod`, `type`, `priority`

- `goalId` or `milestoneId` (mutually exclusive, enforced at DB level)
 - `startDate`, `endDate`
-

4. Actions

What it does: Actions are the atomic, schedulable unit of work in the platform. They can be standalone (independent), project-linked, or gathered (automatically created from Intervals or Routines).

Key features:

- Create with title, scheduled date (`tbd`), priority, estimated time (minutes), and optional start time of day
- Toggle done/not-done
- Link to a project
- Schedule onto a specific date

Action types (by origin):

Type	Description
Standalone	Manually created, no project link
Project-linked	Belongs to a project (<code>projectId</code> set)
Gathered	Auto-created from an Interval or Routine (<code>isGathered=true</code> , <code>sourceType</code> , <code>sourceId</code> , <code>forDate</code> set)

Action fate (disposition after not completing): When an action is not completed on its scheduled day, it must be dealt with in the After-day wizard. Possible fates:

Fate	Meaning
Postponed	Rescheduled to a new date
OutsourceWoo	Delegated — two follow-up actions created
Backlog	Marked as not important (linked actions)
BucketList	Marked as ignore (standalone actions)
PassedArchived	Passed / auto-archived (non-linked gathered, routines)

5. Intervals

What it does: Intervals are recurring activity templates. They define a repeating task that the system automatically generates Actions for (today, tomorrow, day after tomorrow — on each `runActionGathering` call).

Key features:

- Rich recurrence engine:
 - Simple: every N minutes/hours/days/weeks/months/years
 - Weekly with specific days of week (Mon-Sun)
 - Monthly with specific days of month (1-31)
 - Yearly with specific months and optional days of month
 - Specific date list (ad-hoc custom dates, combined with or replacing recurrence)
- Optional `endTime` (stop generating after this date)
- Optional `predictedToDoTime` ("HH:mm" default start time for generated actions)
- Steps: ordered sub-steps for each repetition (drag-reorderable)
- Active/inactive status
- Scope link: optionally linked to a goal, milestone, or project (exclusive)

How gathering works: When `runActionGathering` is called (happens automatically at After-day completion), it:

1. Looks at today, today+1, today+2
2. For each date not yet gathered, checks all active Intervals and Routines
3. For each occurrence on that date, creates a gathered Action with `isGathered=true`, `sourceType`, `sourceId`, `forDate`

- 4. Deduplicates (won't create a duplicate if already gathered for that slot)
- 5. Marks the date's `actionGatheringCompletedAt` on DayState

Linking and After-day behavior:

- Gathered actions from intervals linked to a goal/milestone/project are classified as "linked gathered" — they require mandatory disposition in After-day
- Gathered actions from standalone intervals or routines are "non-linked gathered" — auto-archived if not done

6. Routines

What it does: Routines are daily recurring activities (e.g. morning routine, exercise, journaling). Unlike Intervals, they are not linked to goals/projects and repeat daily based on time-of-day blocks.

Key features:

- Define one or more time-of-day blocks (e.g. "09:00", "20:00") — a gathered action is created per block per day
- Optional `timerDurationMinutes` (for a future focus/timer feature)
- Ordered steps (same as Intervals)
- Active/inactive status and optional end date

Distinction from Intervals:

	Interval	Routine
Linked to goal/project	Yes	No
Custom recurrence	Yes (complex)	No (daily only via time blocks)
After-day disposition	Auto-archived if not done	Auto-archived if not done
Time blocks	Via customRepeatRule	Direct <code>timeOfDayBlocks</code> field

7. Today Module

What it does: The Today module is the primary daily-use interface. It shows all actions scheduled for the current day and is only accessible after completing the Pre-day wizard.

Key features:

- Two sections: **Linked Actions** (project-linked or gathered) and **Standalone Actions**
 - Quick-add a new standalone action inline (title + date + estimated time + time of day)
 - Toggle actions done/not-done
 - Shows scheduled start times and estimated durations
 - Gated: requires Pre-day to be completed before showing the action list
 - "After day" button always available to trigger the After-day wizard
-

8. Pre-day Wizard

What it does: A guided morning ritual to review and prepare the day before starting work. It must be completed before the Today view becomes accessible.

Steps:

1. **(Optional) After-day for yesterday** — if yesterday's After-day was not completed, it runs first
2. **Day overview** — shows all timed actions in chronological order, plus a list of untimed tasks. Overlap warnings shown in amber.
3. **Set times for untimed tasks** — for each action without a start time, user must either: set a time, postpone, outsource, ignore (standalone only), or pass (interval-gathered only). Real-time overlap warnings shown while picking times.
4. **Final overview and begin** — read-only chronological review of the full day. "Begin" button marks `preDayCompletedAt` and unlocks the Today view.

Overlap detection: The system computes time overlaps using `[start, start+estimated)` intervals. Overlaps are flagged visually in amber. If the user submits times that still have overlaps, a confirmation dialog is shown before proceeding.

9. After-day Wizard

What it does: An end-of-day review flow to process unfinished actions and close the day. It must be completed for the previous day before the next day's Pre-day can run.

Steps:

1. **Review mandatory linked actions** — any gathered interval-linked actions that weren't done must be explicitly handled: Postpone (pick new date), Outsource (creates two follow-up actions), Not important (→ Backlog), or Pass (→ PassedArchived)

2. **Review standalone actions** — optional. User can postpone, outsource, ignore (→ BucketList), or delete each unfinished standalone action
3. **Day That Passed** — read-only summary of what was completed that day
4. **Tomorrow review** — preview of tomorrow's scheduled actions (read-only, not shown when embedded inside Pre-day)

On completion:

- `completeAfterDay` mutation sets `afterDayCompletedAt` on `DayState`
- `runActionGathering` is triggered for today/tomorrow/day-after

Auto-archiving: Non-linked gathered actions (routines, standalone intervals) are automatically archived (`PassedArchived`) without requiring user input when After-day opens.

10. Calendar

What it does: A calendar view showing all scheduled actions. Uses `react-big-calendar` under the hood with a custom toolbar and two views: month and week/day list.

Key features:

- Month view with week rows
 - Week/day list view
 - Actions displayed as calendar events with time information
 - Custom event component (`CalendarEvent`)
-

11. Activities (Navigation Section)

The Activities section is the structural home for Goals, Projects, Actions, and Intervals/Routines. Navigation items include:

- Goals list → individual goal management page
 - Projects list
 - Actions list
 - Intervals & Routines list
-

DayState

`DayState` is a per-user, per-date state record that tracks the daily lifecycle:

Field	Meaning
<code>actionGatheringCompletedAt</code>	Gathering ran for this date
<code>preDayCompletedAt</code>	Pre-day wizard completed; Today is accessible
<code>afterDayCompletedAt</code>	After-day wizard completed; next day's Pre-day is unblocked

The daily flow enforces: **After-day (previous day) → Pre-day (today) → Today → After-day (today)**.

What Is and Isn't Built

Built and functional

- Full CRUD for Goals, Goal Groups, Milestones, Projects, Actions, Intervals, Routines
- Goal hierarchy with goal groups and milestone-linked child goals
- Interval recurrence engine (daily, weekly, monthly, yearly, custom dates)
- Action gathering pipeline (today + 2 days ahead)
- Pre-day and After-day wizards with overlap detection and full disposition options
- Today module with linked/standalone action separation
- Calendar view
- Auth (register/login, JWT)
- Priority system

Known missing / in progress

- **Goal DoD clarity check** — the `dod` field exists and is editable, but the structured clarity/quality check (described as a core feature) is not yet implemented
- **Focus Mode** — commented out in TodayPage (`UnderConstruction`)
- **Maintain pillar** — not started (dimension tracking for physical/mental/spiritual)
- **Grow pillar** — not started (EI/Language of Needs, learning habits)
- **Share pillar** — explicitly deferred (shared goals, co-working)
- **Settings page** — placeholder exists (`SettingsPage.tsx`)
- **Tools page** — placeholder exists (`ToolsPage.tsx`)
- **Native apps** — currently web-only (PWA manifest present)